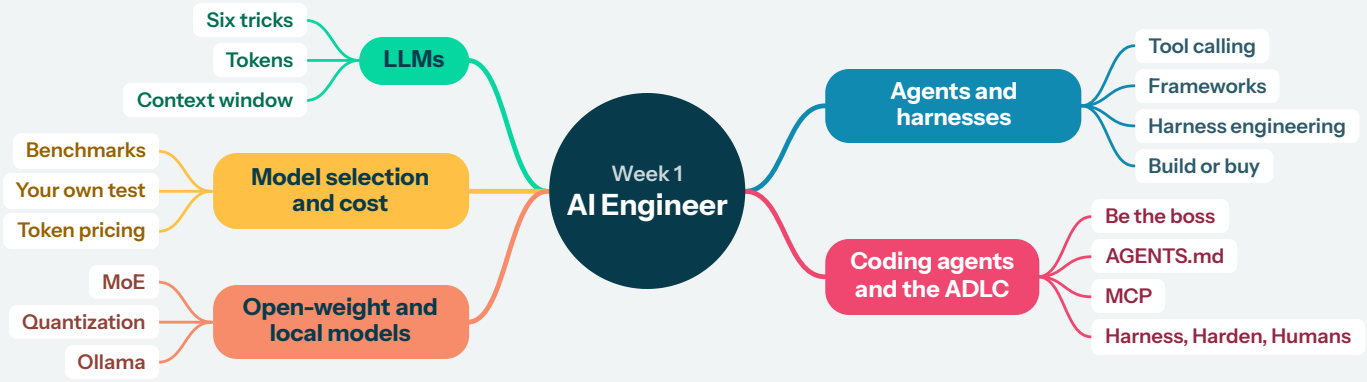
AI FDE: The Forward Deployed Engineer & Agentic AI Architect

Week 1: Become an AI Engineer

SUMMARY SHEET



LLMs

- **Three layers:** the model (GPT, Claude), the platform that runs it (OpenRouter, Bedrock) and the product (ChatGPT, Claude Code)
- **Six tricks from GPT to ChatGPT:** autoregression, chat, reasoning, the illusion of memory, context, tools
- **Stateless:** each call starts fresh, so memory is the history sent again
- **Tokens:** chunks of a few letters. **Context window:** the limit on input plus output
- **Structured outputs:** a Pydantic class, rationale first so the model reasons before it decides
- **METR time horizon:** how long a software task, in human hours, models can complete 50% of the time

Agents and harnesses

- **Agent:** an LLM with tools in a loop to achieve a goal. Agent = Model + Harness
- **Tool calling:** the model asks, and your code runs the tool and returns the result
- **Frameworks** run the loop and the tools: LangChain `create_agent` with `@tool`
- **Fixed workflow?** One LLM call can be faster and cheaper than an agent
- **Context engineering** (2025): instructions, memory, history, retrieved data, tools, prompt, output format
- **Harness engineering** (2026): context, workflow, state, sub-agents, permissions, observability, self-improvement
- **Build or buy:** direct calls, `create_agent`, Deep Agents, Claude Agent SDK
- **Harness patterns:** a goal, a to-do list, sub-agents with fresh context, retries, handoff to a human
- **Self-improvement:** fix mistakes in the harness. **Recursive:** improve how it improves

Model selection and cost

- **Trade off intelligence, speed and cost.** The **Pareto front** is the best quality at each price
- **Benchmarks** such as Artificial Analysis make the shortlist. Test it on the customer's task
- **Score against labels** and compare with a simple baseline, such as always giving the most common answer
- **Reasoning effort** (none to high) costs time and tokens. Test whether it helps
- **Mistakes cost differently:** weigh false positives and false negatives by their cost to the business
- **Accuracy first,** then the cost of the whole business task
- **Token pricing:** input, cached input, output. Reasoning is billed as output
- **OpenRouter:** one model, many providers, which vary in precision and price. Pin them with `order`

Open-weight and local models

- **Open-weight:** downloadable weights. **Open-source:** data and code too
- **Why:** privacy, control and fine-tuning. You still pay for compute and admin
- **Sending data to the labs:** the same security review as any other vendor
- **Dense** uses every parameter. **MoE** uses a few experts per token: 35B-A3B has 3B active, so it's fast, but all 35B must fit
- **Quantization:** lower-precision weights. 8-bit loses little, 4-bit more. **QAT** trains the model for it
- **Memory in GB** ≈ billions of parameters × bits ÷ 8, plus context
- **Distillation:** a smaller model trained to mimic a bigger one
- **Formats:** safetensors, GGUF, MLX
- **Run it:** Ollama or LM Studio on a laptop, vLLM to serve, all OpenAI-compatible
- **Hardware:** memory (what fits) and bandwidth (speed). Try `llmfit`

Coding agents and the ADLC

- **Forms:** IDE (Cursor), extension (GitHub Copilot), terminal (Claude Code, Codex, OpenCode)
- **Harness and model:** Claude Code is the harness, Claude Opus 5.5 the model. Free: OpenCode with OpenCode Zen
- **Be the boss:** you're accountable. Ask hard questions and demand evidence
- **AGENTS.md:** short, with links to detail (progressive disclosure)
- **Practices:** `/clear` often, a plan in phases with success criteria, frequent commits. The repo is the memory
- **Permission modes:** manual, accept edits, plan, auto, bypass
- **MCP:** a standard way to use someone else's tools, like Playwright for a browser
- **ADLC:** building code with agents, or building agents
- **Harness, Harden, Humans:** the agent's platform; checks like linters, types, tests and hooks; the people accountable
- **Review the code,** even when tests pass

The FDE role

- A customer-facing AI engineer, what we used to call an AI solutions architect
- Three disciplines: AI Engineering, AI Architecture and Forward Deployed Engineering. Two kinds of experience: solving business problems and delivering to customers